Jokoa: guía del proyecto

Jokoa es una tienda con tiendas físicas en Bizkaia. Vende y alquila videojuegos, y también vende móviles, ordenadores, tablets y material gaming. Detrás hay cuatro repositorios que trabajan juntos: un backend y tres clientes (web, Android e Ionic). Los tres clientes hablan con la misma API y comparten la misma base de datos.

Repositorio	Para qué sirve	Con qué está hecho
`juegalmi-backend`	La API, la base de datos y la infraestructura en AWS	Symfony 7.4, MariaDB 11.4, JWT, Docker, Terraform, nginx
`jokoa-web`	La tienda web y el panel de administración	React 19, TypeScript, Vite, React Router 7
`jokoa-android`	La app nativa de Android	Java, Volley, Glide, osmdroid
`jokoa-ionic`	La app híbrida que se empaqueta como APK	Ionic 9, Angular 22, Capacitor 8

En producción todo vive en `https://jokoa.duckdns.org`: la web en la raíz y la API bajo `/ws`.

1. Cómo encaja todo

Los clientes mandan JSON por HTTPS y se identifican con un token JWT que dura 8 horas. Cuando caduca, cualquiera de las tres aplicaciones te devuelve a la pantalla de login.

La petición entra por una máquina “bastión”, la única con IP pública. Ahí nginx gestiona el HTTPS con Let’s Encrypt y reenvía el tráfico a la máquina de Symfony, que está en una red privada. Esa segunda máquina ejecuta nginx, PHP-FPM y MariaDB, y sirve también las fotos y la web ya compilada.

- Internet llega al **bastión** (nginx con HTTPS).
- El bastión reenvía a **jokoa-symfony** (10.0.2.10, red privada).
- En jokoa-symfony, `/ws` va a Symfony, `/img` y `/uploads` a las fotos, y todo lo demás a la web de React.
- El bastión sirve además de salto SSH y de NAT, para que las máquinas privadas salgan a Internet sin IP pública.

Dos detalles que conviene saber. No hay IP elástica, para ahorrar crédito: la IP del bastión cambia cada vez que reinicias el laboratorio y DuckDNS la vuelve a actualizar cada 5 minutos. Y la web se sirve desde el mismo dominio que la API, así que en producción no hay problemas de CORS.

Qué puede hacer un cliente

- **Catálogo:** videojuegos, móviles y artículos (portátiles, sobremesa, PC gaming, tablets, consolas, mandos, auriculares, teclados, ratones, monitores y sillas), con ficha técnica, buscador, orden y filtro de stock.
- **Compra:** puede añadir garantía ampliada, elegir tienda de recogida y descargar la factura en PDF.

- **Alquiler:** los videojuegos se alquilan por días y se devuelven desde la propia app.
- **Cuenta:** registro con verificación por correo, perfil, favoritos y un comparador de 2 o 3 productos.
- **Soporte:** tickets de soporte y de reparación. En las reparaciones el cliente acepta o rechaza el presupuesto. Hay además un chat con el administrador y un contador de mensajes sin leer.
- **Asesor de compra con IA:** un chat que busca en el catálogo real y recomienda productos.
- **Administración (solo en la web):** resumen, usuarios, tiendas, fotos, altas y bajas de productos y tickets.
- **Modo oscuro** en las tres aplicaciones.

2. Backend (`juegalmi-backend`)

Qué hay en cada carpeta

- `api/`: el proyecto Symfony (controladores, entidades, servicios, migraciones y fixtures).
- `terraform/`: la red de AWS y las dos máquinas, el bastión (`t3.micro`) y la de Symfony (`t3.small`).
- `bastion/`: la configuración de nginx del bastión y los scripts para sacar el certificado.
- `docker/` y `docker-compose.yml`: PHP-FPM y nginx de la máquina privada.
- `db/`: MariaDB con Adminer y los scripts de mantenimiento.
- `duckdns/`: actualización automática de la IP del dominio.
- `postman/`: la colección de pruebas y el script que la ejecuta.
- `docs/`: el diagrama entidad-relación y los créditos de las imágenes.

Piezas importantes del código

- `WsController` concentra casi toda la API: registro, perfil, catálogos, compras, alquileres, tickets, favoritos y asistente.
- `AdminController` atiende `/ws/admin`; `TicketChatController` el chat de tickets; `VerificacionController` la verificación del correo.
- `Garantias` calcula el precio de la garantía: un 8 % del precio por 12 meses y un 14 % por 24 meses.
- `Factura` genera el PDF; `VerificacionCorreo` crea el token y manda el correo.
- `Asistente` habla con un proveedor de IA compatible con *chat completions* (por defecto Groq). La IA solo puede consultar el catálogo mediante una herramienta de búsqueda, y las tarjetas de producto las monta el servidor con ids reales, así que no puede inventarse productos.
- `UserChecker` impide entrar a las cuentas desactivadas.

Seguridad

Solo son públicos el login, el registro y la verificación del correo; todo lo demás exige token. Hay dos roles, `ROLE_USER` y `ROLE_ADMIN`, y el alta, edición y baja de productos, los tickets y todo `/ws/admin` son solo para administradores.

Hay reglas que el cliente no puede saltarse. El comprador sale siempre del token, no del JSON que envíe. El precio de cada producto y el de la garantía los fija el servidor. El registro está

limitado a 8 por hora y el reenvío del correo a 10 por hora.

Los secretos nunca van en Git. El archivo `api/.env` solo trae valores de ejemplo; lo real vive en `api/.env.local` o `api/.env.prod.local`, en las claves JWT (`api/config/jwt/*.pem`), en `db/.env` y en el token de DuckDNS. Si necesitas cerrar todas las sesiones, `db/rotar-jwt.sh` genera claves nuevas.

Variables de entorno

Variable	Para qué sirve
`DATABASE_URL`	Conexión a MariaDB (la del repositorio es solo un ejemplo)
`JWT_SECRET_KEY`, `JWT_PUBLIC_KEY`, `JWT_PASSPHRASE`	Claves de los tokens; la frase va solo en `.env.local`
`CORS_ALLOW_ORIGIN`	Orígenes permitidos; por defecto solo localhost
`MAILER_DSN`, `MAIL_REMITENTE`, `APP_URL`	Envío de correo y enlaces de verificación
`VERIFICAR_EMAIL`	Con `1` hay que verificar el correo; con `0` las cuentas nuevas entran ya verificadas
`ASISTENTE_API_KEY`, `ASISTENTE_URL`, `ASISTENTE_MODELO`	Asesor con IA; sin clave queda desactivado

Cómo están relacionados los datos

`Producto` es la base común: nombre, descripción, precio, stock, características e imágenes. Cada producto es además un `Videojuego` (con plataforma, género, PEGI y precio de alquiler por día), un `Movil` (marca, almacenamiento y RAM) o un `Articulo` (categoría, marca y montaje).

Alrededor de eso: un `Usuario` tiene compras (con sus líneas), alquileres, tickets (con mensajes), favoritos y ubicaciones. Cada compra puede indicar la `Tienda` donde se recoge. Las líneas de compra guardan el precio del momento y la garantía elegida, para que un cambio de precio posterior no altere compras antiguas.

Un ticket de reparación sigue este camino: el administrador lo deja en `presupuestado` con un importe; el cliente lo acepta (pasa a `en_reparacion`) o lo rechaza (se cierra). No se puede pasar a `en_reparacion` ni a `listo` sin presupuesto aceptado.

El esquema se gestiona con migraciones de Doctrine (`api/src/Migrations`). El diagrama está en `docs/modelo-er.png` y `docs/er-chen.png`.

Scripts de mantenimiento (`db/`)

- `cambiar-claves.sh` y `cambiar-admin.sh`: cambian contraseñas (de MariaDB y de un usuario) sin mostrarlas en pantalla.
- `configurar-correo.sh`: pide el servidor SMTP, envía un correo de prueba y, si llega, activa la verificación.
- `configurar-asistente.sh`: guarda la clave del asesor con IA.
- `copia.sh` y `restaurar.sh`: copia de seguridad cada hora por cron (se conservan 7 días) y restauración. `descargar-copias.ps1` baja las copias a tu PC.

- `rotar-jwt.sh`: rota las claves de los tokens.

Desplegar desde cero

1. **Infraestructura.** Con las credenciales del laboratorio en `~/.aws/credentials`, ejecuta en `terraform/`: `terraform init` y `terraform apply -var "mi_ip=$(curl -s https://checkip.amazonaws.com)"`. Después añade a `~/.ssh/config` los alias `jokoa-bastion` y `jokoa-symfony` (este con `ProxyJump jokoa-bastion`).
2. **Symfony.** En `jokoa-symfony`, levanta MariaDB desde `db/`, lanza `docker compose up -d --build`, instala dependencias con Composer, genera las claves JWT, aplica las migraciones y, solo en un entorno nuevo, carga los fixtures.
3. **Bastión.** Ejecuta `duckdns/instalar.sh`, `bastion/preparar.sh` y `bastion/activar-https.sh`. El certificado se renueva solo.
4. **Web.** Desde `jokoa-web`, `./desplegar.sh` compila y publica.

Con `APP_ENV=prod`, tras cualquier cambio de código hay que ejecutar `php bin/console cache:clear`.

Ojo con el gasto: las dos máquinas consumen aproximadamente 1 $ al día encendidas. Cuando termines, pulsa **End Lab**; y si quieres borrar todo, `terraform destroy`. Las credenciales del Learner Lab caducan al apagarlo.

Probar la API

Importa `postman/JuegAlmi-API.postman_collection.json` en Postman, o ejecuta `python postman/probar.py`. Deberías ver 69 de 69 peticiones correctas. Necesitas las variables `JOKOA_PASSWORD` y `JOKOA_ADMIN_PASSWORD`. Cada ejecución crea y borra datos de prueba y registra un cliente nuevo (`cliente<número>@juegalmi.com`), así que mejor lanzarla sobre datos de demostración. Para desactivar esos clientes: `UPDATE usuario SET activo=0 WHERE email LIKE 'cliente%@juegalmi.com'`.

Cuentas de demostración

En producción solo existen `ekaitz@jokoa.com` (administrador), `almi@gmail.com` y `sergio@gmail.com` (clientes). Sus contraseñas no están en el repositorio. En un entorno nuevo, los fixtures crean `admin@juegalmi.com`, `ane@juegalmi.com` y `jon@juegalmi.com`, con contraseña `demo1234`.

3. Web (`jokoa-web`)

Es una sola aplicación de React que incluye la tienda y el panel de administración (`/admin`, solo para usuarios con rol de administrador).

- `api.ts` es el cliente de la API. Guarda el token en `localStorage`, comprueba si ha caducado y avisa a la aplicación cuando eso ocurre.
- El estado global está en pequeños contextos: sesión (`auth.tsx`), carrito, favoritos, comparador, avisos y tema claro u oscuro.
- `pages/` contiene las pantallas de la tienda y `admin/` las del panel: resumen, productos, pedidos, tickets, tiendas y usuarios.
- `components/` agrupa las piezas reutilizables, como la tarjeta de producto, el modal, el chat de tickets y el chat del asesor.
- Los estilos están en `styles/`, con los colores y tamaños centralizados en `tokens.css`.

Rutas. Sin sesión: `/`, `/login`, `/registro` y `/verificar`. Con sesión: `/juegos`, `/moviles`, `/ordenadores`, `/tablets`, `/gaming`, `/buscar`, `/favoritos`, `/comparar`, `/producto/:tipo/:id`, `/carrito`, `/pedidos`, `/perfil`, `/tiendas` y `/soporte`. Solo administradores: `/admin/*`.

Trabajar con ella:

- `npm install` instala las dependencias.
- `npm run dev` arranca Vite y redirige `/ws`, `/img` y `/uploads` a la API real.
- `npm run build` comprueba los tipos y genera `dist/`.
- `./desplegar.sh` se ejecuta en la máquina de Symfony: compila y copia el resultado a `~/juegalmi/web`. Da por hecho que el repositorio está en `~/juegalmi-web` y que Node está en `~/herramientas/node/bin`.

4. App Android (`jokoa-android`)

Aplicación nativa en Java (`com.jokoa.app`) para Android 7.0 o superior (minSdk 24, targetSdk 35), con Material, Volley para la red, Glide para las imágenes y osmdroid para el mapa.

- **Pantallas (activities):** splash, login, registro, principal con navegación inferior, detalle de producto, soporte y comparador.
- **Fragmentos de la pantalla principal:** inicio, productos, galería, fotos, asesor, mapa, tiendas, pedidos y perfil.
- `red/Api.java` tiene una única cola de Volley y la dirección base `https://jokoa.duckdns.org/ws`; añade el token a cada petición. Si la API responde que la sesión no es válida, borra el token y te lleva al login. `red/Sesion.java` guarda el token en SharedPreferences.
- `datos/` contiene los modelos y el estado local (productos, favoritos, comparador).
- La tienda tiene cinco secciones: Juegos, Móviles, Ordenadores, Tablets y Gaming. La ficha permite comprar, alquilar (juegos) y añadir garantía (móviles, tablets y ordenadores).

Compilar: necesitas Android Studio y JDK 17, y un archivo `local.properties` con `sdk.dir=...` (no se sube a Git). Después, `./gradlew assembleDebug` deja el APK en `app/build/outputs/apk/debug/`. Si quieres apuntar a otro servidor, cambia `Api.BASE`.

5. App Ionic (`jokoa-ionic`)

La misma tienda, escrita con Ionic y Angular y empaquetada para Android con Capacitor (`com.jokoa.app.ionic`).

- `core/` reúne la lógica: servicio de API, sesión, guard de rutas, interceptor que añade el token y gestiona la caducidad, y los servicios de favoritos, comparador, avisos y asesor.
- `pages/` tiene las pantallas: splash, login, registro, las cinco pestañas (tienda, asesor, tiendas, pedidos y perfil), detalle y soporte.
- `theme/` guarda el diseño: `variables.scss` con los colores y `jokoa.scss` con los componentes.
- Hay tres entornos: desarrollo y web de producción usan `/ws`, y el APK usa la URL completa, porque la app no está en el dominio de la API.
- Capacitor tiene activado `CapacitorHttp`, de modo que las peticiones salen por el sistema y no por el WebView. Así no hay problemas de CORS.

Ejecutar y generar el APK:

- `npm install` instala las dependencias.
- `npx ng serve` abre la app en `http://localhost:4200`.
- `npx ng build --configuration app` construye la versión para el APK.
- `npx cap add android` y `npx cap sync android` preparan el proyecto nativo; después se abre la carpeta `android/` en Android Studio.

Una diferencia a tener en cuenta: la app Ionic cubre Juegos y Móviles, mientras que la web y la app nativa también tienen Ordenadores, Tablets y Gaming.

Una errata: los README de Android e Ionic mencionan una “web Angular”, pero la web real es de React.

6. Referencia de la API

Base: `https://jokoa.duckdns.org/ws`. Todo en JSON y con la cabecera `Authorization: Bearer <token>`. Una escritura correcta responde `{"resultado":"ok","id":N}`. Un error responde `{"resultado":"error","mensaje":"..."}` con un código: 400 (datos incorrectos), 403 (sin permiso o recurso ajeno), 404 (no existe) o 409 (conflicto, por ejemplo sin stock).

Las rutas siguen un patrón: `GET /x`, `GET /x/{id}`, `POST /x/add`, `PUT /x/update/{id}` y `DELETE /x/delete/{id}`.

Cuenta

Método y ruta	Quién	Qué hace
`POST /login`	Público	Con `{email, password}` devuelve el token. Las cuentas desactivadas no entran
`POST /registro`	Público	Crea un cliente (máximo 8 por hora)
`POST /verificar` y `/verificar/reenviar`	Público	Confirma el correo o reenvía el enlace
`GET /perfil`	Usuario	Datos del usuario del token
`PUT /perfil/update`	Usuario	Cambia nombre, apellidos, teléfono o contraseña

Catálogo

Método y ruta	Quién	Qué hace
`GET /plataformas`, `/departamentos`, `/tiendas`	Usuario	Listas auxiliares
`GET /videojuegos`, `/moviles`, `/articulos` y `/{id}` de cada uno	Usuario	Listado y ficha. Los artículos admiten filtro por categoría

Método y ruta	Quién	Qué hace
`POST .../add`, `PUT .../update/{id}`, `DELETE .../delete/{id}` en `/videojuegos`, `/moviles` y `/articulos`	Admin	Altas, ediciones y bajas

Categorías de artículo: portatil, sobremesa, pc_gaming, tablet, consola, mando, auriculares, teclado, raton, monitor y silla.

Compras y alquileres

Método y ruta	Quién	Qué hace
`GET /compras`	Usuario	Tus compras (el administrador ve todas)
`POST /compras/add`	Usuario	Crea una compra; descuenta stock
`GET /compras/{id}/factura`	Dueño o admin	Factura en PDF
`GET /alquileres`	Usuario	Tus alquileres (el administrador ve todos)
`POST /alquileres/add`	Usuario	Con `{videojuego_id, dias}` resta una copia
`PUT /alquileres/devolver/{id}`	Dueño o admin	Repone el stock y registra la devolución

Ejemplo de compra. `tienda_id` y `garantia_meses` (12 o 24) son opcionales:

```
{
  "tienda_id": 1,
  "lineas": [ { "producto_id": 12, "cantidad": 1, "garantia_meses": 24 } ]
}
```

Tickets, favoritos y asistente

Método y ruta	Quién	Qué hace
`GET /tickets`	Usuario	Tus tickets (el administrador ve todos)
`POST /tickets/add`	Usuario	`{departamento_id, asunto, descripcion, dispositivo, tipo}`; `tipo` es `soporte` o `reparacion`
`PUT /tickets/presupuesto/{id}`	Dueño	`{aceptado: true o false}`
`PUT /tickets/update/{id}` y `DELETE /tickets/delete/{id}`	Admin	Cambiar estado o presupuesto, o eliminar

Método y ruta	Quién	Qué hace
`GET /tickets/no-leidos`	Usuario	Mensajes sin leer
`GET` y `POST /tickets/{id}/mensajes`	Dueño o admin	Leer o escribir en el chat del ticket
`GET /favoritos`, `PUT` y `DELETE /favoritos/{id}`	Usuario	Ver, añadir o quitar favoritos
`POST /asistente`	Usuario	Envía el historial de la conversación y devuelve `{respuesta, productos[]}`

Administración (solo `ROLE_ADMIN`)

Método y ruta	Qué hace
`GET /admin/resumen`	Métricas del panel
`GET /admin/usuarios`, `POST /admin/usuarios/add`, `PUT /admin/usuarios/update/{id}`	Gestión de usuarios: rol, contraseña, activar o desactivar
`POST /admin/imagenes`	Sube fotos de producto (hasta 3 MB por petición)
`POST /admin/tiendas/add`, `PUT .../update/{id}`, `DELETE .../delete/{id}`	Gestión de tiendas

Esta referencia sale de leer los controladores, no de probar cada endpoint. Para el formato exacto de los cuerpos que no aparecen aquí, mira la colección de Postman o el código de `api/src/Controller/`.